Category	Requirement
Infrastructure	The system shall run on a Commit AI-POC AWS account with all required services provisioned.
Infrastructure	The system shall be deployable to the client's own AWS account via documented deployment scripts.
Golden Data	The client shall provide a ground truth (GT) dataset of ≥20 videos across all 4 video types (≥5 per type, ≥10 for job site), each with a per-video GT JSON containing: video metadata (class, road_type, weather, traffic, speed), hazards with timestamps/severity, and per-frame annotations (objects with class/bbox/distance, lane polygons, road signs, traffic signals). Client shall also provide 4 example config JSONs (one per video type) in their application's expected output format.
UI	The MVP UI shall allow users to: upload video(s), upload a single image, specify an S3 output path, view processing status, and download/view results.
Model Selection	For each detection category (motorcycle, signs, construction), at least 2 pre-trained models shall be benchmarked on the golden dataset, and a winner selected.
Detection	The pipeline shall detect motorcycles in video frames and output: class label, bounding box and estimated distance.
Detection	The pipeline shall detect lane markings with colored overlays and polygon output. If multi-lane confidence is low, fallback to driver's lane only.
Detection	The pipeline shall detect and classify road signs/traffic signals into types: speed, stop, yield, warning, info, traffic light.
Detection	The detection pipeline shall maintain or improve performance under rainy/low-visibility conditions compared to PoC baseline.
Detection	The system shall classify each video's road type as one of: highway, city, town, rural.

Detection	The system shall track objects across consecutive frames and flag a hazard when an object enters the driver's lane.
Classification	The system shall classify each input video into one of 4 types: Reactivity, Educational/Q&A, Hazard Detection, Job Site — and generate a type-specific config JSON saved to S3.
Job Site	The system shall detect and classify job site objects in video frames with a comprehensive label set.
Job Site	The user shall be able to upload a single image via the UI and receive detection results (annotated image + JSON).
Output UI	The user shall be able to specify a custom S3 output folder path via the UI that exsists in the client's AWS account. All outputs (JSONs, annotated frames) save to that path, or a default path if empty.
Output UI	The system shall support batch upload of multiple videos with async processing, per-video S3 subfolders, and completion notification.
Testing	An automated evaluation report shall compare MVP vs. PoC performance on the golden dataset across all detection categories.
Testing	E2E test: Upload 3 reactivity/braking videos → correct classification → detection on all frames → config JSON saved to S3.
Testing	E2E test: Upload 3 educational/Q&A videos → correct classification → Q&A config with ≥3 questions per video → saved to S3.
Testing	E2E test: Upload 3 hazard videos → correct classification → cross-frame tracking → hazard config with risk levels → saved to S3.
Testing	E2E test: Upload 3 construction videos → correct classification → distinct object types detected → config saved to S3.
Testing	E2E test: Run pipeline on 5 rainy/low-visibility clips — no crashes, weather field correct in output.
Testing	E2E test: Batch upload 10 mixed-type videos → all processed async, classified correctly, outputs saved.

Documentation	AI developer shall deliver: GitHub repo with README (setup, dependencies, env vars, run & deploy instructions), pipeline flow diagram, LLM model swap guide, known limitations doc (≥10 items), and operational guides (usage, accuracy, cost, debugging).
Documentation	DevOps shall deliver: AWS cloud architecture diagram (all services, data flows, integrations) and AI pipeline architecture diagram (detection models, LLM classifier, config generator).

1 requirements

Acceptance Criteria (Testable)
1. AI developer can log in and access required AWS services (S3, Lambda, SQS, Bedrock, etc.). 2. IAM roles and permissions are configured for all pipeline components. 3. No permission errors when running the pipeline end-to-end.
1. Run deployment script on a clean client AWS account — pipeline deploys without manual steps. 2. All services start successfully (health check returns 200). 3. Upload a test video — pipeline processes it end-to-end.
1. ≥20 videos delivered covering all 4 types: reactivity_braking, qa_educational, hazard_detection, job_site_detection (≥5 each, ≥10 job site). 2. Each video has 1 GT JSON with: filename, video_description, video_class, road_type, weather, traffic, speed, hazards[], and frames[]. 3. Each frame entry contains: frame_number, timestamp_sec, objects[] (class, description, distance, center, bbox), lanes[] (lane_id, type, polygon), road_signs[] (label, bbox), traffic_signals[] (label, bbox). 4. Object classes use allowed values: car, truck, bus, motorcycle, bicycle, pedestrian, construction_worker, cone, barrier, heavy_equipment, construction_sign, fencing, debris, animal, other. 5. Lane types are 'ego_lane' or 'other_lane' with polygon coordinates. 6. Road sign labels from: speed limit, stop, yield, warning, construction. Traffic signal labels from: red, yellow, green, flashing_red, flashing_yellow, off, other. 7. Hazard severity from: low, medium, high, critical. Video class from: reactivity_braking, qa_educational, hazard_detection, job_site_detection. 8. 4 example config JSONs delivered (one per video type) in client's application format. 9. Evaluation script loads GT dataset and computes precision/recall per category without errors.
1. User can upload a single video and see a progress/status indicator. 2. User can upload a single image and receive annotated output. 3. User can enter a custom S3 path or leave blank for default. 4. User can upload multiple videos in batch mode. 5. All controls are functional in the deployed environment.
1. Benchmark exists with precision/recall per model per category. 2. ≥2 models tested per category.
1. On golden dataset motorcycle frames: recall ≥80%. 2. Output JSON per frame contains: object_type='motorcycle', bbox[x,y,w,h], distance_m. 3. Annotated frame visually shows bounding boxes on detected motorcycles.
1. On GT highway frames: lane polygon IoU ≥80%. 2. On 5 urban frames: driver's lane detected with IoU ≥80%. 3. Output JSON: lane_id, lane_polygons[], type. 4. Each lane rendered in a distinct color.
1. On GT sign frames: precision ≥80%. 2. Output JSON per detection: sign_label, bbox. 3. Annotated frame shows labeled bounding boxes per sign.
1. Run detection on 10 rainy validation frames. 2. ≥30% improvement over PoC baseline on same frames.
1. On the provided test videos : ≥80% correct classification. 2. Output JSON contains road_type field with value in {highway, city, town, rural}.

1. Output JSON includes hazard when an object enters the driver's lane.
1. On 20 test videos (5 per class): ≥80% correct classification. 2. Config JSON generated per video matches client schema. 3. Config JSON saved to the designated S3 bucket. 4. JSON contains video_class field.
1. On 10 job site GT videos: precision ≥80% on average. 2. Output JSON: object_label, bbox, distance, description. 3. On 5 non-job-site highway frames: zero false-positive construction objects.
1. Upload PNG or JPG → annotated image returned with bounding boxes + labels. 2. Accompanying JSON lists detected objects. 3. Upload invalid file type (e.g., .txt) → UI shows error message.
1. Enter valid path '/client/project-a/results' → all outputs at that S3 prefix. 2. Leave path empty → outputs go to default path. 3. Path persists across multiple uploads in one session.
1. Upload 5+ videos → all queued for processing. 2. All complete without manual intervention. 3. Each video's output in its own S3 subfolder. 4. User receives completion notification in UI. 5. One failed video does not block others.
1. Report table: Category | PoC Precision | MVP Precision |. 2. Evaluation runs against client-provided GT JSONs (per-video annotations with bbox, class, lane polygons, sign labels, etc.). 3. Motorcycle recall ≥80%, Lane IoU ≥80%, Signs precision ≥80%, Construction precision ≥80%. 4. Report generated by a reproducible script (no manual steps). 5. Results broken down per video type (reactivity, educational, hazard, job site).
1. All 3 classified as 'Reactivity' with 80% precision (2 videos are classified correctly). 2. Detection output exists for each frame. 3. Config JSON validates against client schema.
1. All 3 classified as 'Educational' with 80% precision (2 videos are classified correctly).. 2. Q&A config contains ≥3 relevant questions per video. 3. All referenced objects exist in detection output. 4. Config at correct S3 path.
1. All 3 classified as 'Hazard' with 80% precision (2 videos are classified correctly).. 2. Config includes risk levels and scenario descriptions. 3. Config saved to S3.
1. All 3 classified as 'Construction' with 80% precision (2 videos are classified correctly).. 2. construction object types in config. 3. Config saved to S3.
1. Pipeline completes on all 5 clips (no crashes). 2. Output JSON weather field = 'rain' or 'low_visibility'.
1. All 10 queued in SQS and processed. 2. ≥80% classification accuracy across all types. 3. All configs + annotations saved to S3.

1. Clone repo + follow README → environment sets up without errors. 2. Pipeline flow diagram delivered (PNG + editable source) showing all stages from upload to S3 output. 3. LLM swap guide enables a developer to change models in <30 min. 4. Known limitations doc covers ≥10 items with description, impact, and workaround. 5. Operational guides delivered for: usage, accuracy optimization, cost optimization, pipeline debugging, output quality debugging.
1. Cloud architecture diagram includes: S3, Lambda, SQS, Bedrock, Rekognition, Fargate, VPC. 2. AI pipeline diagram shows: detection models, LLM classifier, config generator. 3. Both delivered as PNG + editable source (draw.io or Mermaid).

Owner	SoW Time estimation (Days)	End date	Notes
DevOps	3		
DevOps	3		
Product	1		Done, waiting for the client to provide data
Product + AI	2		
AI	2		
AI	2		
AI	1		
AI	1		
AI	1		
AI	1		

AI	1		
AI	2		
AI	1		
AI	1		
AI	0.5		
AI	1		
AI	2		
Product	1		
Product	1		
Product	1		
Product	1		
Product	1		
Product	1		

AI			
DevOps			